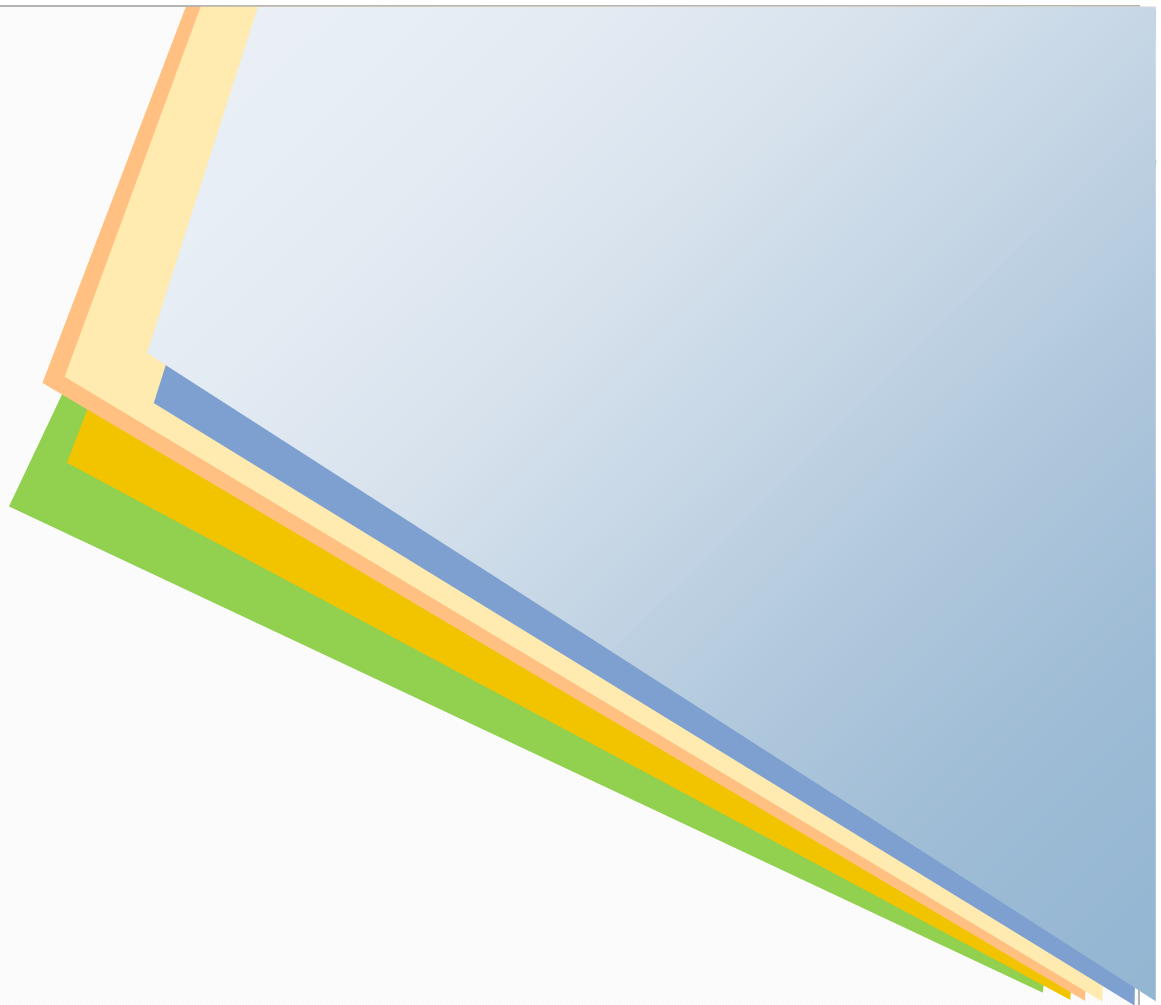




DS413 Data Exploration and Visualization

PROJECT DESCRIPTION

Information Systems Department

2023



Data Exploration and Visualization

Group members:

Students Names	Student ID
Alaa Alshuaibi	442005420
Monerah Almobarak	442002988
Nada Alotaibi	442003374
Sarah Aljuhani	442005104
Sarah Altaweel	442000786



- **Data extraction:**

The dataset utilized for our analysis is sourced from Kaggle. The dataset with 70,000 rows encompasses a wealth of information about the Olympics, including historical data, details about host cities, participating nations, and athletic achievements. This dataset serves as a valuable resource for uncovering patterns, trends, and insights that contribute to a nuanced understanding of the Olympics

The dataset comprises several key columns, each playing a role in providing a holistic perspective on the Olympic Games. The columns include:

1. **ID:** A unique identifier for each entry in the dataset.
2. **Name:** The name of the participating athlete.
3. **Sex:** The gender of the athlete.
4. **Age:** The age of the athlete at the time of the event.
5. **Height:** The height of the athlete.
6. **Weight:** The weight of the athlete.
7. **Team:** The name of the team to which the athlete belongs.
8. **NOC:** National Olympic Committee code, representing the country.
9. **Games:** Information about the specific Olympic Games, including the year and season.
10. **Year:** The year in which the Olympic Games were held.
11. **Season:** The season of the Olympic Games.
12. **City:** The host city for a particular edition of the Olympics.
13. **Sport:** The type of sport involved in a specific event.
14. **Event:** The specific event or competition within a sport.
15. **Medal:** The type of medal awarded (Gold, Silver, Bronze, or None).

These columns collectively provide a comprehensive dataset that allows for a detailed exploration of the Olympic Games over time, offering insights into historical trends, athlete performances, and the broader context of this global sporting event [1].

- **Data Preprocessing:**

The data preprocessing phase is crucial to ensure the quality and reliability of our dataset. In this step, we first dropped the initial column ID, as it did not contribute to the analysis. Next, we conducted a comprehensive check for missing values across all columns. The following columns were identified as having missing values:

- **Age:** 2732 missing values
- **Height:** 16254 missing values
- **Weight:** 17101 missing values
- **Medal:** 60310 missing values



Given the substantial number of missing values in the 'Medal' column, a pragmatic approach was taken to maintain completeness in our analysis. Recognizing that these missing values likely indicate instances where athletes did not win a medal, we filled these gaps with the label 'No Medal'. This decision ensures that the absence of a medal is explicitly captured in our dataset, allowing for a more nuanced exploration of athletic achievements.

Following this, we addressed missing values in the 'Age', 'Height', and 'Weight' columns by removing the corresponding rows. Additionally, we checked for duplicate rows, and it is noteworthy that no duplicate rows were found in the dataset.

To enhance the dataset's integrity, outliers were addressed through a two-step process. First, we focused on the 'Year' column, creating a box plot to visualize its distribution. Outliers were then identified and removed based on the Interquartile Range (IQR) method.

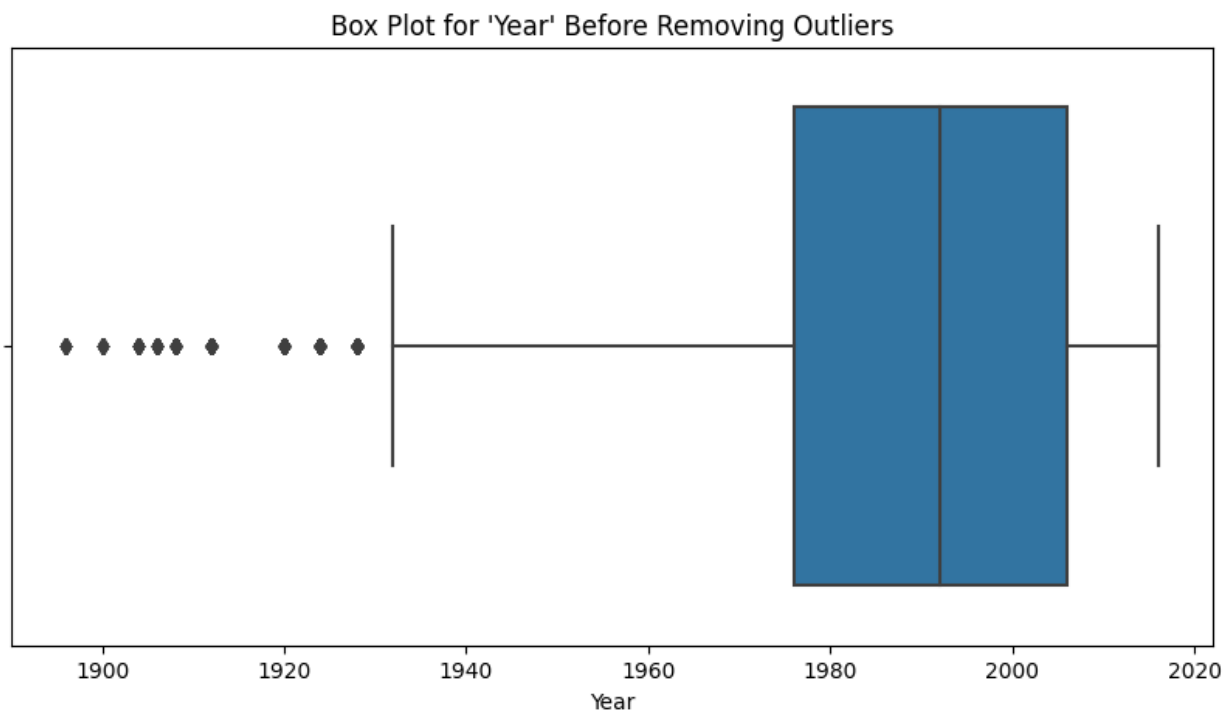


Figure 1: Box Plot for 'Year' Before Removing Outliers

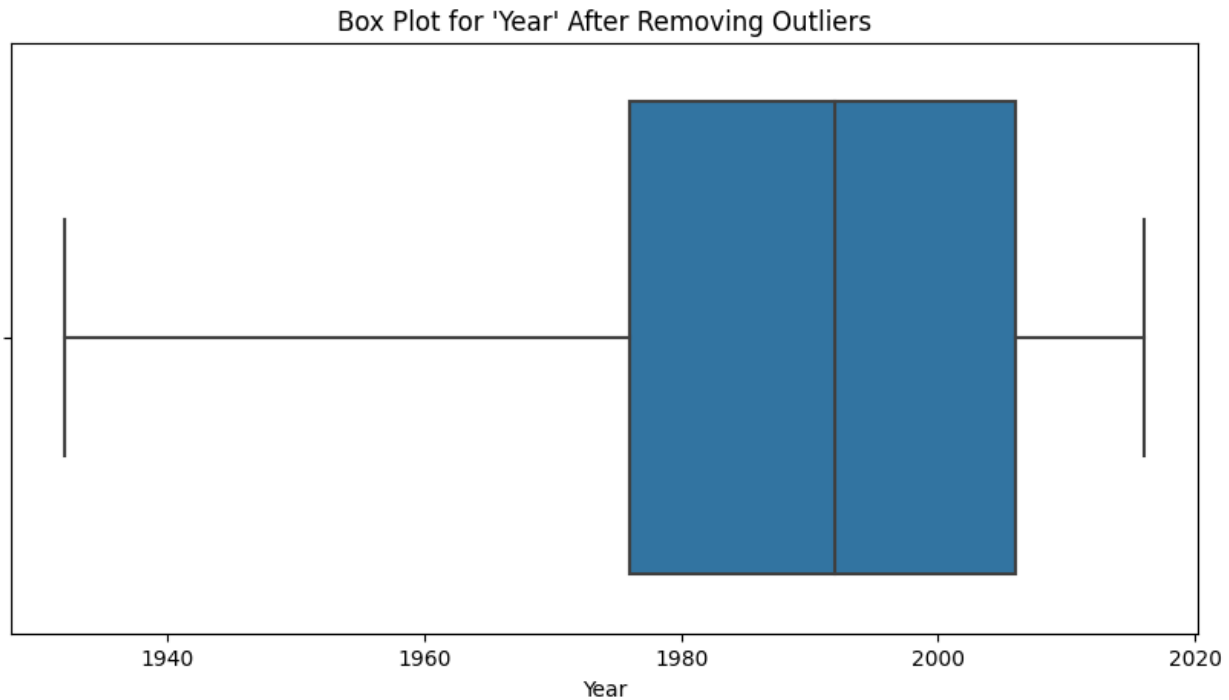


Figure 2: Box Plot for 'Year' After Removing Outliers

Subsequently, box plots were generated for the 'Age', 'Height', and 'Weight' columns to visualize their distributions before and after outlier removal. Outliers in these columns were filtered using the IQR method, ensuring that extreme values did not distort our analyses.

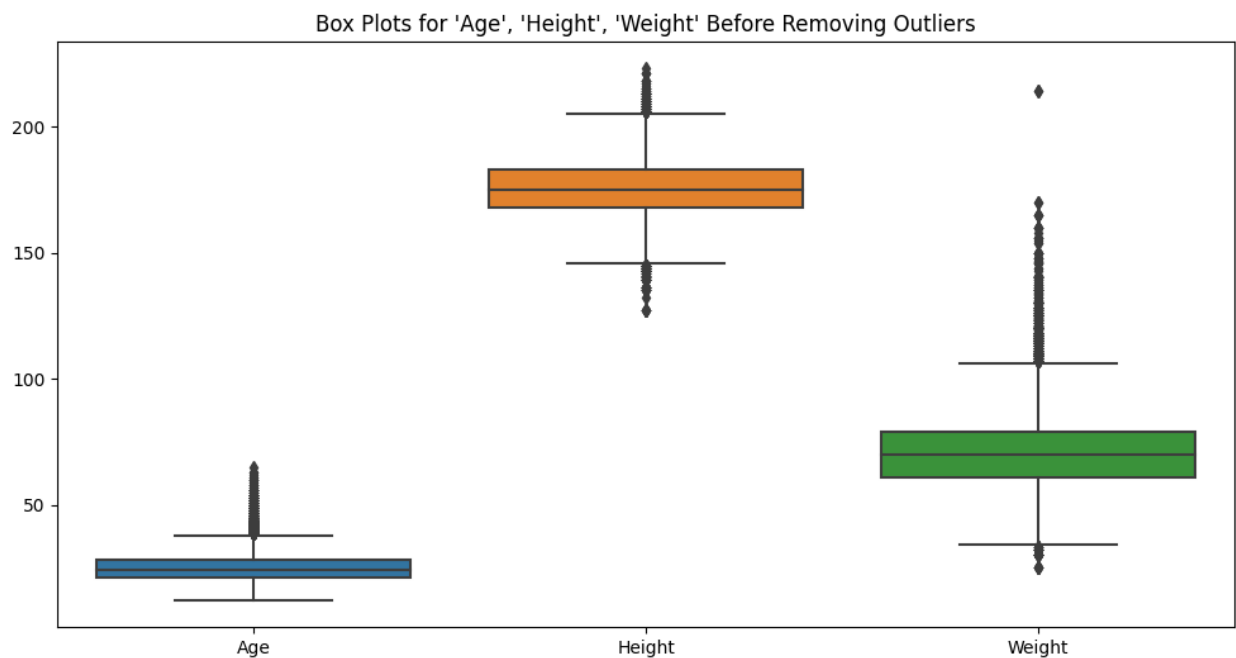


Figure 3: Box Plot for 'Age', 'Height', and 'Weight' Before Removing Outliers

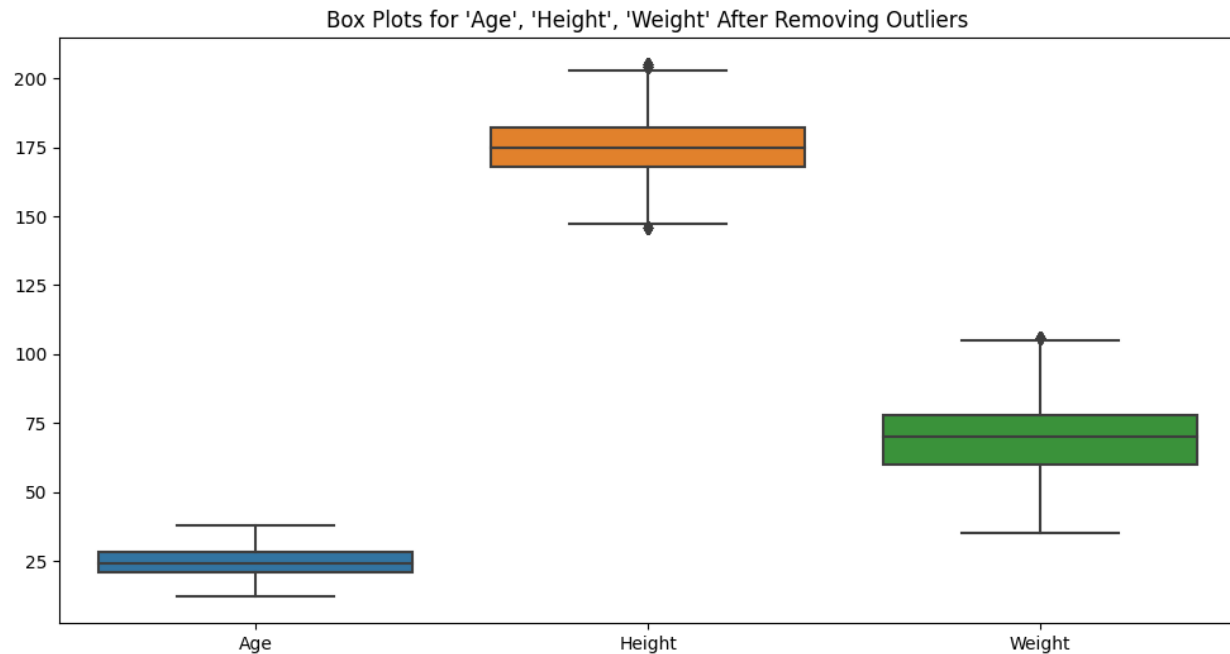


Figure 4: Box Plot for 'Age', 'Height', and 'Weight' After Removing Outliers

The resulting dataset, now free from duplicate entries and outliers, is well-prepared for insightful analyses, providing a robust foundation for exploring trends and patterns related to Olympic Games data [2].

• Data Exploration:

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 49211 entries, 0 to 69999
Data columns (total 14 columns):
#   Column  Non-Null Count  Dtype  
---  -
0    Name    49211 non-null   object 
1    Sex      49211 non-null   object 
2    Age      49211 non-null   float64
3    Height   49211 non-null   float64
4    Weight   49211 non-null   float64
5    Team     49211 non-null   object 
6    NOC      49211 non-null   object 
7    Games    49211 non-null   object 
8    Year     49211 non-null   int64  
9    Season   49211 non-null   object 
10   City     49211 non-null   object 
11   Sport    49211 non-null   object 
12   Event    49211 non-null   object 
13   Medal    49211 non-null   object 
dtypes: float64(3), int64(1), object(10)
memory usage: 5.6+ MB
```

Figure 5: Olympic_Data_Info

A pandas Data Frame with 14 columns and 49211 rows, containing information about Olympic athletes, such as their name, sex, age, height, weight, team, NOC, Games, year, season, city, sport, event, and medal.

	Year	Age	Height	Weight
count	49211.000000	49211.000000	49211.000000	49211.000000
mean	1990.651602	24.593688	175.351588	70.019691
std	18.070465	4.703656	10.080288	12.650303
min	1932.000000	12.000000	146.000000	35.000000
25%	1976.000000	21.000000	168.000000	60.000000
50%	1992.000000	24.000000	175.000000	70.000000
75%	2006.000000	28.000000	182.000000	78.000000
max	2016.000000	38.000000	205.000000	106.000000

Figure 6: Statistical Summary

We use the `displays()` function to calculate the statistical summary for the columns Year, Age, Height, and Weight of the Data Frame. It provides information like the average, minimum, maximum, and percentiles for each column, helping understand the central tendency, distribution, and potential outliers of the data.

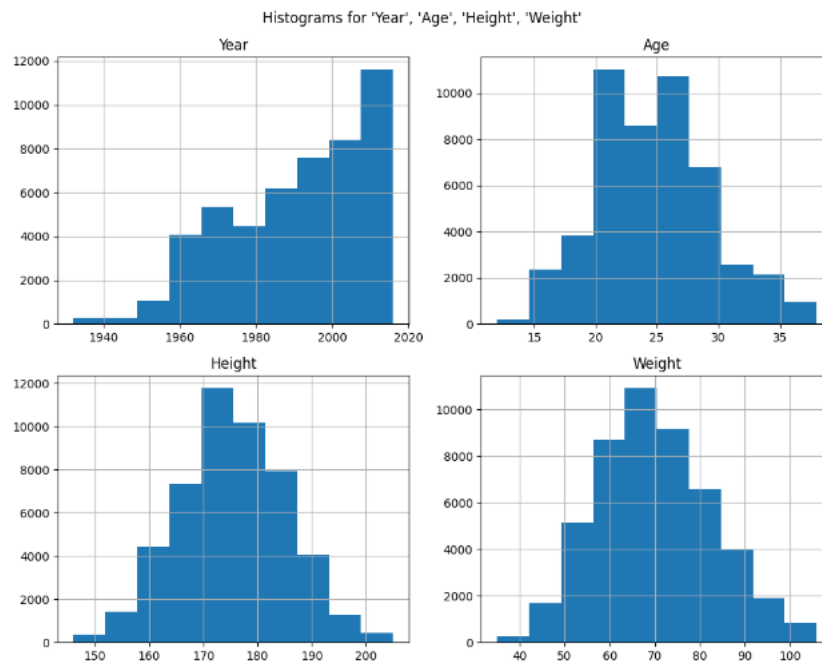


Figure 7: Histograms for 'Year', 'Age', 'Height', and 'Weight'.

The plot shows the distributions of people by age, height, and weight over time. The histograms show that the distribution of people by age has remained relatively stable over time, with a median age of around 25. The distribution of people by height has shifted to the right over time, with the median height increasing by about 5 cm since 1940. The distribution of people by weight has also shifted to the right over time, with the median weight increasing by about 10 kg since 1940.

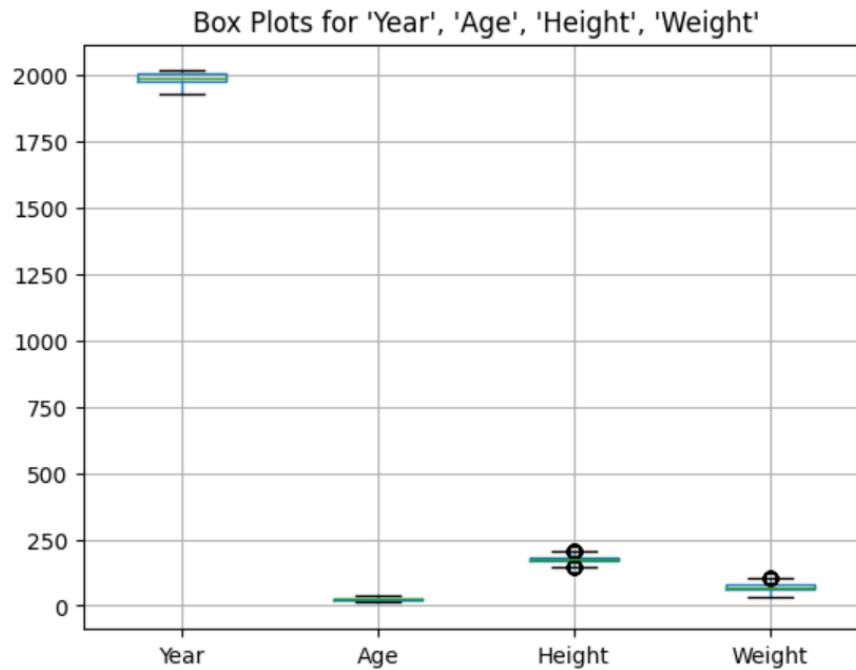


Figure 8: Box Plots for 'Year', 'Age', 'Height', and 'Weight'

This box plot shows the distribution of age, height, and weight for Olympic athletes, grouped by year. The median age of athletes has remained relatively stable over time, while the median height and weight have increased slightly.

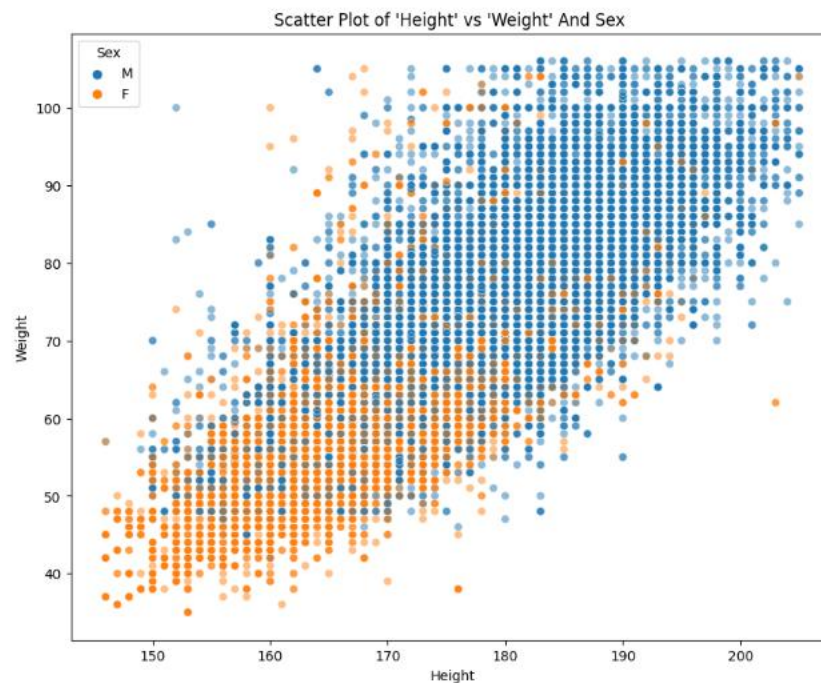


Figure 9: Scatter Plot of 'Height' vs 'Weight' And Sex.

The scatter plot illustrates the relationship between height and weight in a dataset, using colors to distinguish between male and female data points. Each point represents an individual, with height on the x-axis and weight on the y-axis. Transparent points reveal overlapping data, highlighting the prevalence of males. The legend clarifies gender categories, facilitating the interpretation of the height and weight distribution. This visualization effectively communicates patterns, emphasizing gender-based distinctions in the dataset.

Pair Plot for 'Year', 'Age', 'Height', 'Weight', by Medal

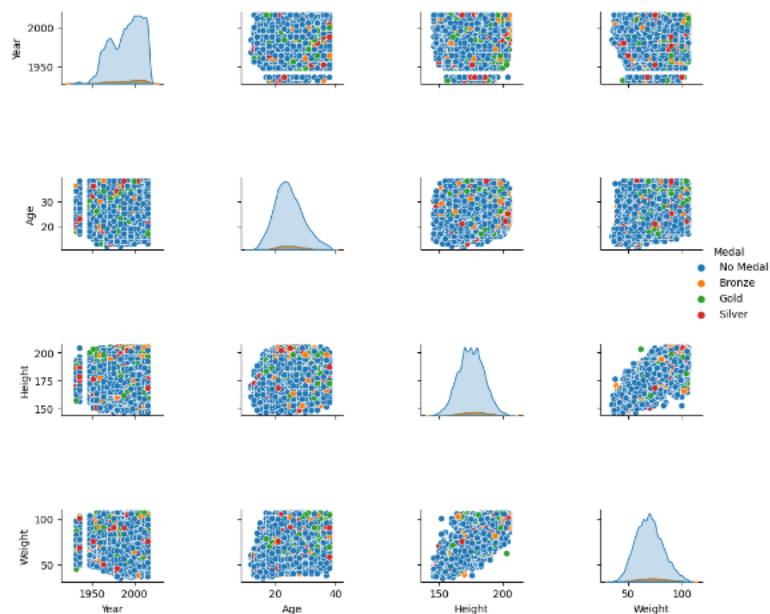


Figure 10: Pair Plot for 'Year', 'Age', 'Height', 'Weight', by Medal

The pair plot visually examines the relationships between 'Year,' 'Age,' 'Height,' 'Weight,' and 'Medal' in Olympic athletes. There are five subplots, each color-coded by the type of medal won. Notable trends emerge: age generally increases with time, height tends to rise over the years, and weight typically follows age. However, considerable variability exists within these trends, showcasing diverse physical characteristics among athletes. The pair plot underscores the absence of a universal profile for Olympic medalists, emphasizing the uniqueness and individuality of competitors in terms of age, height, and weight. The visualizations effectively capture the intricate interplay of these factors within the dataset.

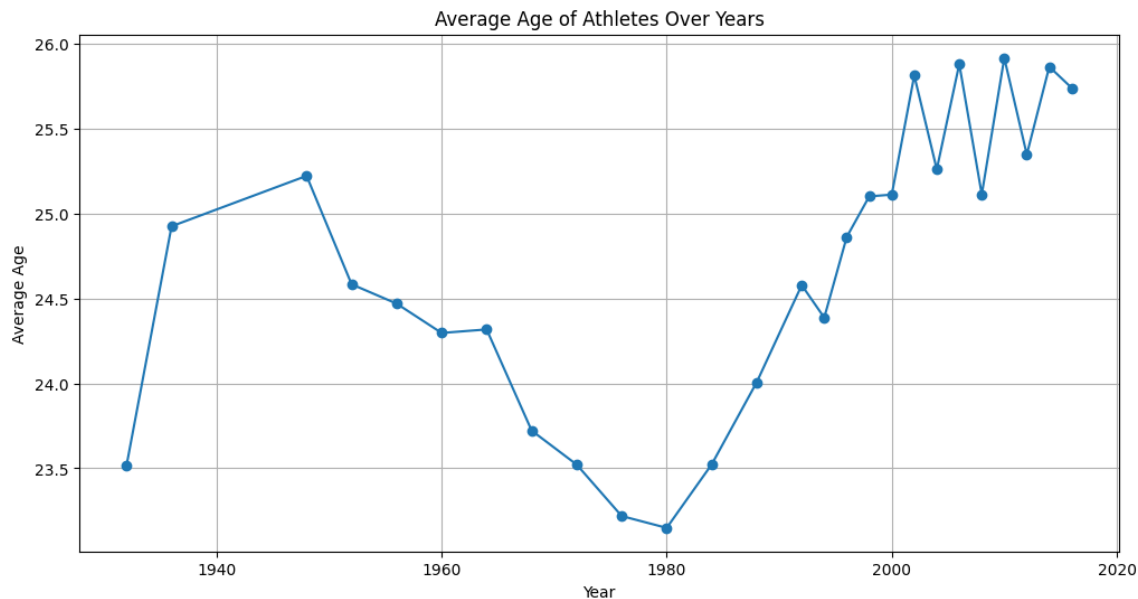


Figure 11: Average Age of Athletes Over Years

The line graph illustrates the trend in the average age of athletes from the 1930s to about 2020. It shows a fluctuating pattern where the average age initially rose, fell to a low in the late 1970s, and then climbed to its peak in the early 2000s. The graph, marked with points for each year, provides a visual narrative of how athletes' average ages have changed over time, indicating a trend towards older average ages in recent years.

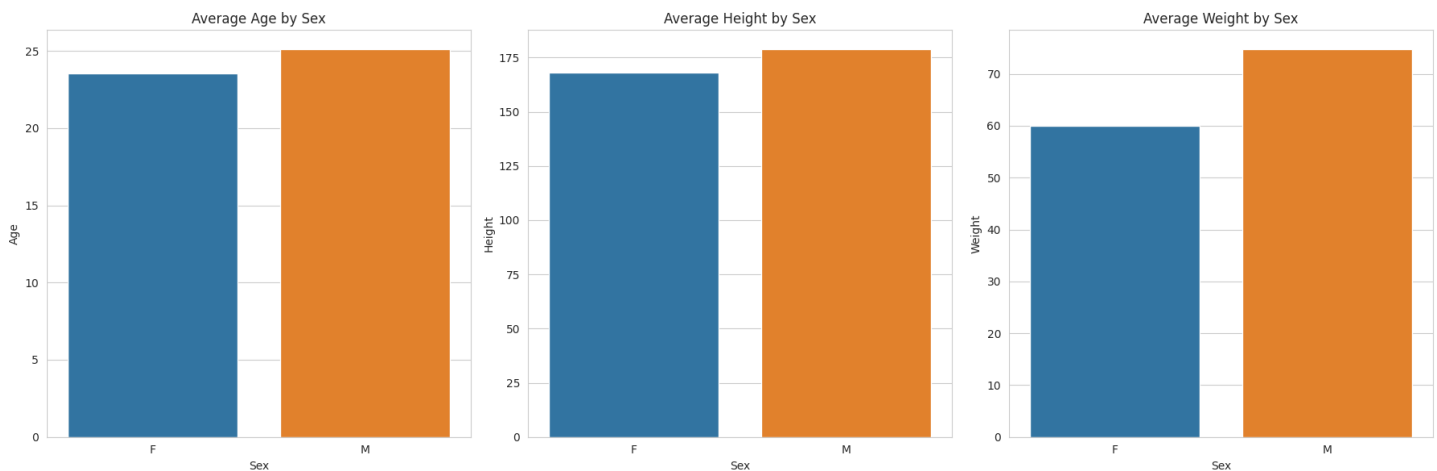


Figure 12: Comparison of Average Age, Height, and Weight by Sex.

The set of three bar graphs depict average age, height, and weight by sex. The first graph shows that the average age of males is slightly higher than that of females. The second graph reveals that males are taller on average than females. The third graph demonstrates that males also have a higher average weight than females. Each graph uses distinct colors for the sexes, aiding in quick visual comparison, with a clean white grid background enhancing readability. These graphs collectively provide a clear comparison of physical attributes by sex.

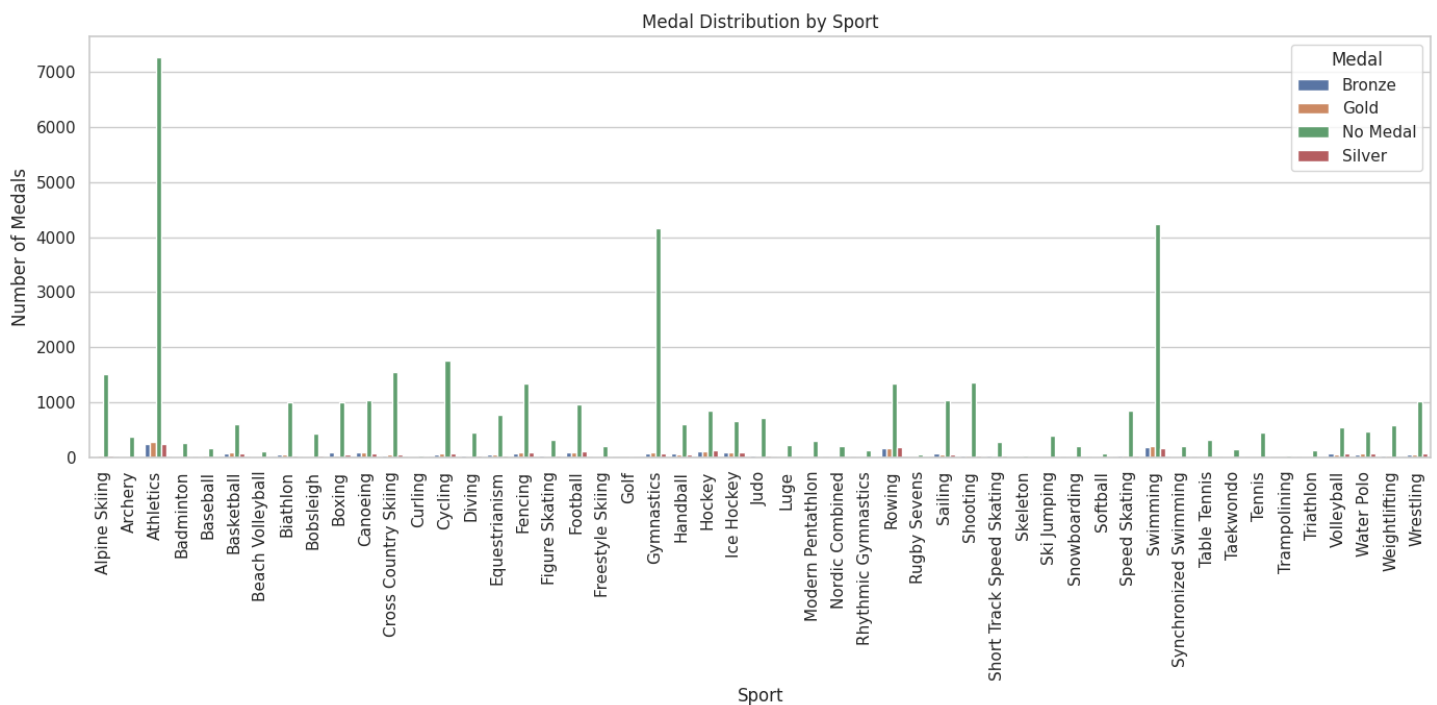


Figure 13: Medal Distribution by Sport

The bar chart titled "Medal Distribution by Sport" displays the number of medals awarded in various sports. The sports are listed along the x-axis, and the number of medals is on the y-axis. Different colored bars represent bronze, gold, silver, and no medals, with a legend categorizing the medal types. Some sports, like Athletics and Swimming, show a high number of medals, especially gold, while others have fewer medals. The chart efficiently conveys the distribution and count of medals across a range of sports, making it easy to

compare the prevalence of medal-winning outcomes in each sport. The x-axis labels are rotated for clarity, preventing overlap and ensuring that each sport's name is readable.

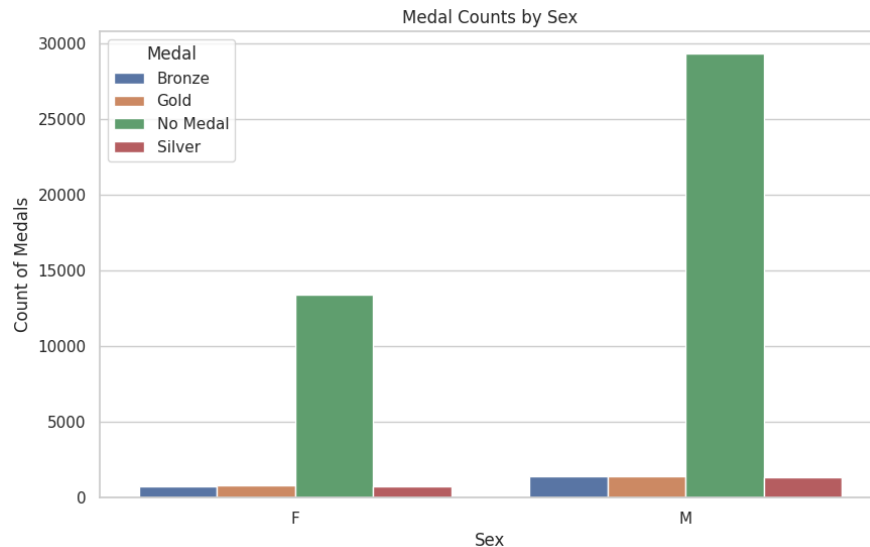


Figure 14: Medal Counts by Sex

The bar chart titled "Medal Counts by Sex" compares the distribution of Olympic medals between females (F) and males (M). Different colors represent the types of medals won: bronze, gold, silver, and no medal. The chart indicates that males have won a significantly higher number of medals across all categories, with gold being the most common, followed by silver and bronze. The legend provides a clear key to interpret which colors correspond to each medal type. This visualization succinctly presents the disparity in medal counts between genders at the Olympic Games.

• Data Visualization:

We used the Plotly python library to perform interactive data visualizations. We used plots such as, boxplot, histogram, bar chart, scatterplot, and geomap. [3]



Figure 15: Interactive Plot for 'Age', 'Height' and 'Weight'

Figure 15 shows the interactive boxplot we created that shows a statistical summary of our three variables Age, Height, and Weight. We employed a Plotly library to make it interactive, where if you hover your mouse on any variable you get the information a boxplot normally gives, such as max, min, first quarter, and third quarter.

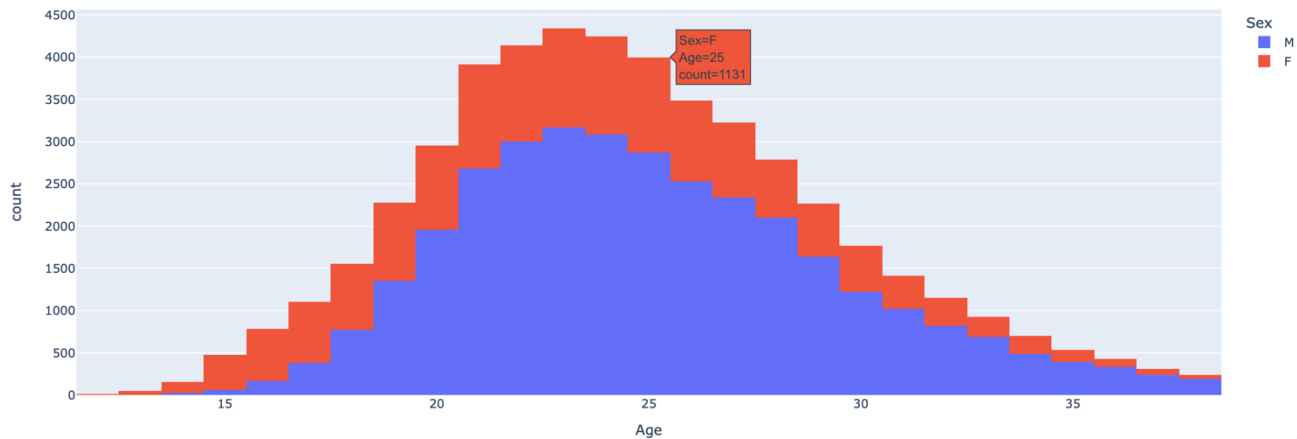


Figure 16: Interactive Plot for 'Age' and 'Sex'

In Figure 16, an interactive histogram created using the Plotly library is displayed. The histogram represents the number of participants categorized by their age and separated by gender. The plot offers interactivity, allowing us to hover over data points and view specific information associated with each point.

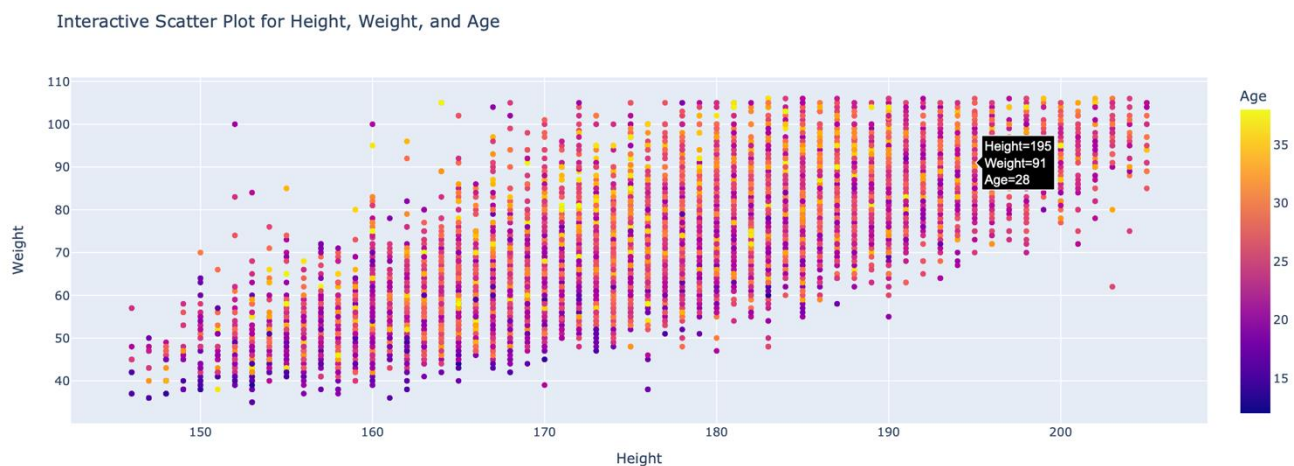


Figure 17: Interactive Plot for 'Height', 'Weight', and 'Age'

Figure 17 displays an interactive scatterplot using Plotly. It reveals a linear relationship between height and weight of Olympic participants, with a majority of young individuals. Hovering over data points provides interactive details of height, weight, and age.

3D Scatter Plot for Height, Weight, and Age

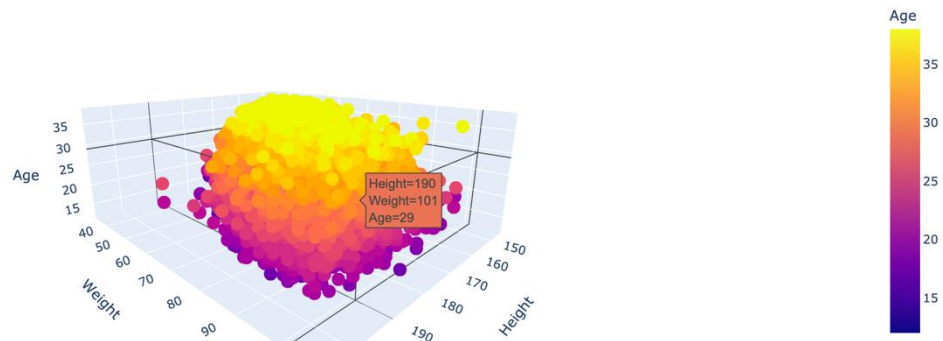


Figure 18: Interactive 3D Plot for Height', 'Weight', and 'Age'

Figure 18 presents an interactive 3D scatter plot that highlights the relationship between height, weight, and age. Created using Plotly, the plot allows zooming and hovering over data points to view specific information associated with each point.

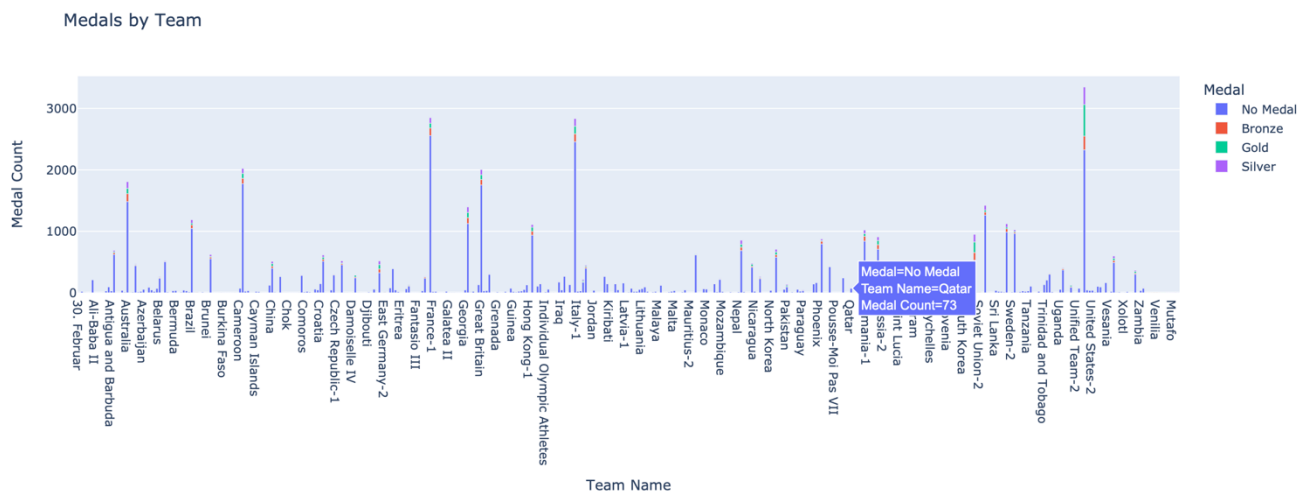


Figure 19: Interactive Plot for 'Medals' by 'Team Name'

Illustrated in Figure 19 is an interactive bar chart generated through the utilization of the Plotly library. This graphical representation provides insightful insight into the performance of various teams concerning their acquired medals. Each individual bar within the chart corresponds to a distinct team, showcasing the quantitative distribution of medals across four distinct categories.

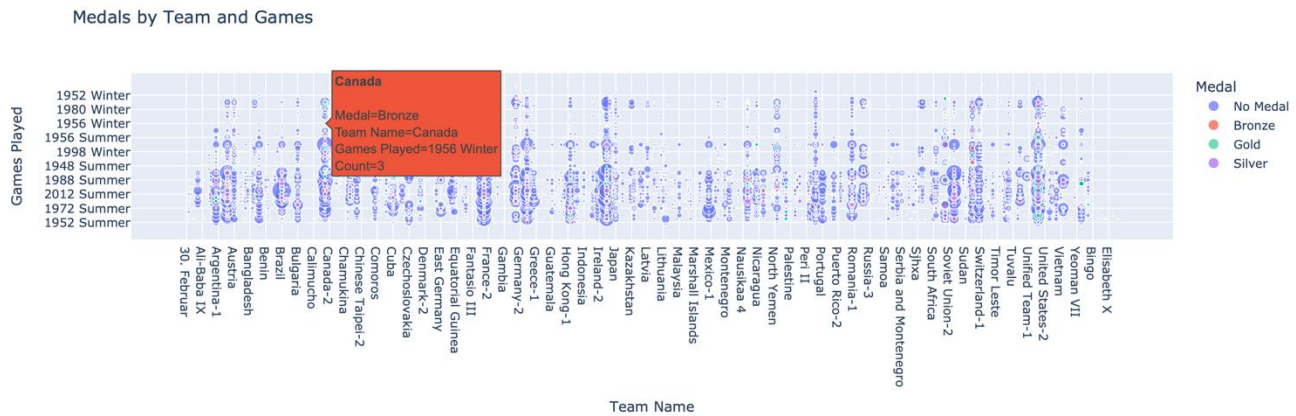


Figure 20: Interactive Plot for 'Medals' by 'Team Name' and 'Games Played'

Illustrated in Figure 20 is an interactive scatter plot generated through the utilization of the Plotly library, portraying the performance trends of various teams over time. The graphical representation highlights the interrelation among pertinent variables, namely 'Team Name,' 'Games Played' (corresponding to the year and month), and 'Medal.' This visualization serves as a facilitative tool for comprehending the performance dynamics of each team within specified temporal intervals.

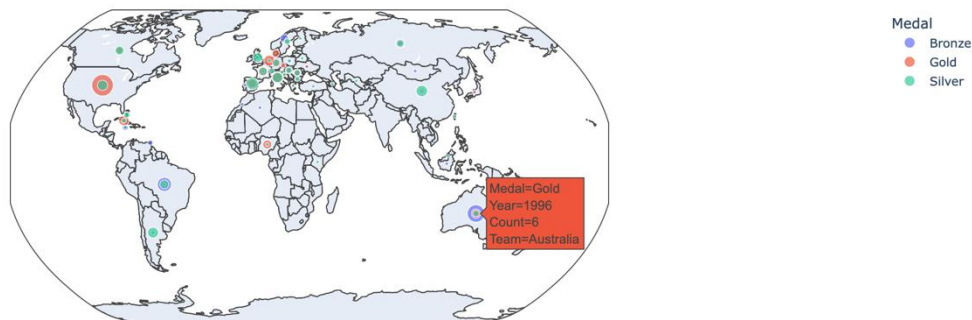


Figure 21: Interactive Plot for 'Medal' and 'Team Location'

Figure 21 presents an interactive Geomap created using Plotly, offering a visual representation of team locations. The plot provides essential details such as team names, associated medal achievements, corresponding years, and the aggregate medal count. This graphical tool facilitates a comprehensive overview of each team's regional distribution.



- **References:**

- [1] B. BISWAS. [Online]. Available: https://www.kaggle.com/datasets/bhanupratapbiswas/olympic-data?select=dataset_olympics.csv.
- [2] S. K. Mukhiya and U. Ahmed, Hands-On Exploratory Data Analysis with Python, Packt Publishing, 2020.
- [3] "Modern Analytic Apps for the Enterprise - Plotly," [Online]. Available: <https://plotly.com/>.

- **Appendix:**

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from scipy.stats import zscore

import seaborn as sns

data = pd.read_csv('/content/dataset_olympics.csv')

data

# Drop the first column

data = data.drop(data.columns[0], axis=1)

data.head

# Check for missing values in each column

missing_values = data.isna().sum()

# Display columns with missing values and their respective counts

columns_with_missing_values = missing_values[missing_values > 0]

if not columns_with_missing_values.empty:

    print("Columns with missing values and their counts:")
```




```
print(columns_with_missing_values)

else:

    print("No missing values found in the dataset.")

#Fill missing values in 'Medal' with a specific label
data['Medal'].fillna('No Medal', inplace=True)

# Display the number of rows before removing missing values
print("\nNumber of rows before removing missing values:", len(data))


# Remove rows with missing values in 'Age', 'Height', and 'Weight' and keep the dataset name as 'data'
data.dropna(subset=['Age', 'Height', 'Weight'], inplace=True)


# Display the number of rows after removing missing values
print("Number of rows after removing missing values:", len(data))

# Check for duplicate rows
duplicate_rows = data[data.duplicated()]

if not duplicate_rows.empty:

    print("Duplicate rows found:")

    print(duplicate_rows)


# Drop the duplicate rows
data = data.drop_duplicates()


print("Deleted the duplicate rows.")

else:
```



```
print("No duplicate rows found in the dataset.")

def plot_boxplot(data, column, title):

    plt.figure(figsize=(10, 5))

    sns.boxplot(x=data[column])

    plt.title(title)

    plt.show()

# Print box plot for 'Year' before removing outliers

plot_boxplot(data, 'Year', "Box Plot for 'Year' Before Removing Outliers")

# Calculate the IQR for 'Year'

Q1_year = data['Year'].quantile(0.25)

Q3_year = data['Year'].quantile(0.75)

IQR_year = Q3_year - Q1_year

# Define a function to filter outliers based on the IQR for 'Year'

def filter_outliers_year():

    lower_bound_year = Q1_year - 1.5 * IQR_year

    upper_bound_year = Q3_year + 1.5 * IQR_year

    return (data['Year'] >= lower_bound_year) & (data['Year'] <= upper_bound_year)

# Apply the filter for 'Year'

outliers_filter_year = filter_outliers_year()

# Print the number of rows before removing outliers for 'Year'
```



```
print(f'Number of rows before removing outliers for 'Year': {len(data)}')

# Remove rows with outliers for 'Year'
data = data[outliers_filter_year]

# Print box plot for 'Year' after removing outliers
plot_boxplot(data, 'Year', "Box Plot for 'Year' After Removing Outliers")

# Print the number of rows after removing outliers for 'Year'
print(f'Number of rows after removing outliers for 'Year': {len(data)}')

# Function to create box plots
def plot_boxplot(data, columns, title):
    plt.figure(figsize=(12, 6))
    sns.boxplot(data=data[columns])
    plt.title(title)
    plt.show()

# Print box plots for 'Age', 'Height', and 'Weight' before removing outliers
plot_boxplot(data, ['Age', 'Height', 'Weight'], "Box Plots for 'Age', 'Height', 'Weight' Before Removing Outliers")

# Calculate the IQR for 'Age', 'Height', and 'Weight'
Q1_columns = data[['Age', 'Height', 'Weight']].quantile(0.25)
Q3_columns = data[['Age', 'Height', 'Weight']].quantile(0.75)
IQR_columns = Q3_columns - Q1_columns
```



```
# Define a function to filter outliers based on the IQR for 'Age', 'Height', and 'Weight'
```

```
def filter_outliers_columns():
```

```
    lower_bound_columns = Q1_columns - 1.5 * IQR_columns
```

```
    upper_bound_columns = Q3_columns + 1.5 * IQR_columns
```

```
    return (
```

```
        (data['Age'] >= lower_bound_columns['Age']) & (data['Age'] <= upper_bound_columns['Age']) &
```

```
        (data['Height'] >= lower_bound_columns['Height']) & (data['Height'] <=
upper_bound_columns['Height']) &
```

```
        (data['Weight'] >= lower_bound_columns['Weight']) & (data['Weight'] <=
upper_bound_columns['Weight'])
```

```
    )
```

```
# Apply the filter for 'Age', 'Height', and 'Weight'
```

```
outliers_filter_columns = filter_outliers_columns()
```

```
# Print the number of rows before removing outliers for 'Age', 'Height', and 'Weight'
```

```
print(f"Number of rows before removing outliers for 'Age', 'Height', 'Weight': {len(data)}")
```

```
# Remove rows with outliers for 'Age', 'Height', and 'Weight'
```

```
data = data[outliers_filter_columns]
```

```
# Print box plots for 'Age', 'Height', and 'Weight' after removing outliers
```

```
plot_boxplot(data, ['Age', 'Height', 'Weight'], "Box Plots for 'Age', 'Height', 'Weight' After Removing Outliers")
```



```
# Print the number of rows after removing outliers for 'Age', 'Height', and 'Weight'

print(f'Number of rows after removing outliers for 'Age', 'Height', 'Weight': {len(data)}')


# Additional information

data.info()

# Value counts

value_counts = data['Medal'].value_counts()

print("Value counts for 'Medal':")

print(value_counts)

# Statistical summary using describe()

summary = data[['Year', 'Age', 'Height', 'Weight']].describe()

print(summary)

# Visualize data using histograms

data[['Year', 'Age', 'Height', 'Weight']].hist(figsize=(10, 8))

plt.suptitle("Histograms for 'Year', 'Age', 'Height', 'Weight'")

plt.tight_layout()

plt.show()

# Visualize data using box plots

data[['Year', 'Age', 'Height', 'Weight']].boxplot()

plt.title("Box Plots for 'Year', 'Age', 'Height', 'Weight'")

plt.show()

# Visualize data using scatter plots

# Assuming 'data' is a pandas DataFrame with the columns 'Height', 'Weight', and 'Sex'.

plt.figure(figsize=(10, 8))

sns.scatterplot(data=data, x='Height', y='Weight', hue='Sex', alpha=0.5)
```



```
plt.title("Scatter Plot of 'Height' vs 'Weight' And Sex ")

plt.xlabel("Height")

plt.ylabel("Weight")

plt.legend(title='Sex') # Add a legend to differentiate colors

plt.show()

# Visualize data using pair plot


import seaborn as sns

import matplotlib.pyplot as plt


# Set the figure size before creating the pairplot

plt.figure(figsize=(30, 30))


# Create the pair plot

pair_plot = sns.pairplot(data[['Year', 'Age', 'Height', 'Weight', 'Medal']], hue='Medal')


# Set the title for the pair plot

pair_plot.fig.suptitle("Pair Plot for 'Year', 'Age', 'Height', 'Weight', by Medal", size=30)


# Adjust the layout with more padding and a higher top margin

plt.tight_layout(pad=7.0)

plt.subplots_adjust(top=0.9) # You might need to adjust this value


# Show the plot

plt.show()
```



```
# Convert 'Year' to datetime format if it's not already
data['Year'] = pd.to_datetime(data['Year'], format='%Y')

# Group the data by 'Year' and calculate the mean age
age_trend = data.groupby(data['Year'].dt.year)['Age'].mean()

# Plot the trend
plt.figure(figsize=(12, 6))
plt.plot(age_trend.index, age_trend.values, marker='o') # Added a marker for each data point
plt.title('Average Age of Athletes Over Years')
plt.xlabel('Year')
plt.ylabel('Average Age')
plt.grid(True)
plt.show()

# Group by 'Sex' and calculate mean for 'Age', 'Height', and 'Weight'
segmented_data = data.groupby('Sex')[['Age', 'Height', 'Weight']].mean()

# Set the aesthetic style of the plots
sns.set_style("whitegrid")

# Initialize the matplotlib figure
f, axes = plt.subplots(1, 3, figsize=(18, 6), sharex=True)

# Create a barplot for 'Age'
sns.barplot(x=segmented_data.index, y='Age', data=segmented_data, ax=axes[0])
```



```
axes[0].set_title('Average Age by Sex')

# Create a barplot for 'Height'
sns.barplot(x=segmented_data.index, y='Height', data=segmented_data, ax=axes[1])
axes[1].set_title('Average Height by Sex')

# Create a barplot for 'Weight'
sns.barplot(x=segmented_data.index, y='Weight', data=segmented_data, ax=axes[2])
axes[2].set_title('Average Weight by Sex')

# Finalize the plot
plt.tight_layout()
plt.show()

# Group by 'Sport' and count how many medals of each type were won
medal_distribution = data.groupby(['Sport', 'Medal']).size().unstack(fill_value=0)

# Set the aesthetics for the plot
sns.set(style="whitegrid")

# Create a figure and a set of subplots
plt.figure(figsize=(14, 7))

# Convert the "medal_distribution" dataframe into "long-form" for seaborn
medal_distribution_long = medal_distribution.reset_index().melt(id_vars='Sport', var_name='Medal',
value_name='Count')
```




```
# Create a barplot

sns.barplot(x='Sport', y='Count', hue='Medal', data=medal_distribution_long)


plt.title('Medal Distribution by Sport')

plt.xlabel('Sport')

plt.ylabel('Number of Medals')

plt.xticks(rotation=90) # Rotate the x labels so they don't overlap

plt.legend(title='Medal')

plt.tight_layout() # Adjust the layout to make room for the x-axis labels

plt.show()

# Make sure that 'Sport' is a category if you had not done this before

data['Sport'] = data['Sport'].astype('category')


# Now perform the groupby operation and unstack

medal_distribution = data.groupby(['Sport', 'Medal']).size().unstack(fill_value=0)


# Reset index to make 'Sport' a column again

medal_distribution = medal_distribution.reset_index()


# Now plot with Matplotlib

import matplotlib.pyplot as plt

medal_distribution.plot(kind='bar', stacked=True, figsize=(14, 7), x='Sport')

plt.title('Medal Distribution by Sport')
```



```
plt.xlabel('Sport')

plt.ylabel('Number of Medals')

plt.xticks(rotation=90) # Rotate the x labels so they don't overlap

plt.legend(title='Medal')


plt.tight_layout() # Adjust the layout to make room for the x-axis labels

plt.show()

# Assuming 'data' is your DataFrame

medals_by_sex = data.groupby(['Sex', 'Medal']).size().reset_index(name='Count')


# Create a barplot

plt.figure(figsize=(10, 6))

sns.barplot(x='Sex', y='Count', hue='Medal', data=medals_by_sex)


plt.title('Medal Counts by Sex')

plt.xlabel('Sex')

plt.ylabel('Count of Medals')

plt.legend(title='Medal')

plt.show()

!pip install plotly

import plotly.express as px


def plot_boxplot_interactive(data, columns, title):

    fig = px.box(data, y=columns, title=title)

    fig.show()
```



```
plot_boxplot_interactive(data, ['Age', 'Height', 'Weight'], "Box Plots for 'Age', 'Height', 'Weight' Before  
Removing Outliers")  
  
fig = px.histogram(data, x="Age", color="Sex", hover_data=data.columns)  
  
fig.show()  
  
import plotly.express as px  
  
# Assuming your DataFrame is named 'data'  
  
fig = px.scatter(data, x='Height', y='Weight', color='Age', title='Interactive Scatter Plot for Height, Weight,  
and Age')  
  
fig.show()  
  
import plotly.express as px  
  
# Assuming your DataFrame is named 'data'  
  
fig = px.scatter_3d(data, x='Height', y='Weight', z='Age', color='Age', title='3D Scatter Plot for Height,  
Weight, and Age')  
  
fig.show()  
  
team_medal_counts = data.groupby(['Team', 'Medal']).size().reset_index(name='Count')  
  
fig = px.bar(team_medal_counts, x='Team', y='Count', color='Medal',  
             title='Medals by Team', labels={'Team': 'Team Name', 'Count': 'Medal Count'})  
  
fig.show()  
  
!pip install dash  
  
import dash  
  
from dash import dcc, html  
  
import plotly.express as px  
  
import pandas as pd
```



```
# Assuming your DataFrame is named 'data' and the columns for Team, Medal, and Games are 'Team',
'Medal', and 'Games' respectively

team_medal_games = data.groupby(['Team', 'Medal', 'Games']).size().reset_index(name='Count')

app = dash.Dash(__name__)

app.layout = html.Div([

    dcc.Graph(

        figure=px.scatter(team_medal_games, x='Team', y='Games', color='Medal',

                           size='Count', hover_name='Team',

                           labels={'Team': 'Team Name', 'Games': 'Games Played'},

                           title='Medals by Team and Games')

    )

])

if __name__ == '__main__':

    app.run_server(debug=True)

import pandas as pd

import holoviews as hv

from holoviews import opts

# Initialize the holoviews extension

hv.extension('bokeh')

# Assuming 'data' is your DataFrame
```



```
scatter = hv.Scatter(data, ['Height', 'Weight', 'Age'])

# Customize the plot if needed

scatter.opts(
    opts.Scatter(color='Age', cmap='viridis', width=600, height=400, tools=['hover'],
        xlabel='Height', ylabel='Weight', title='Interactive Scatter Plot: Height vs Weight by Age')
).redim.range(Height=(data['Height'].min(), data['Height'].max()),
    Weight=(data['Weight'].min(), data['Weight'].max()))

data_sorted = data.sort_values('Year')

team_medals_won_counts = data_sorted.groupby(['Team', 'Medal',
    data_sorted['Year'].dt.year]).size().reset_index(name='Count')

pd.to_datetime(team_medals_won_counts['Year'], errors='ignore')

team_medals_won_counts = team_medals_won_counts[team_medals_won_counts['Medal'] != 'No Medal']

team_medals_won_counts

fig = px.scatter_geo(team_medals_won_counts, locations="Team", locationmode='country names',
    size="Count", color="Medal", animation_frame="Year", projection="natural earth")

fig.update_geos(showcountries=True)

fig.show()
```